



(Company No.: 198301005860)

الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونُزْ بَرَسِيَّتِيْ اِسْلَامُ اَنْتَا رَا بَعْثًا مِّلِّيْسِيَا

Garden of Knowledge and Virtue

MECHATRONICS SYSTEM INTEGRATION : MCTA 3203

LABORATORY REPORT

SECTION 1

TITLE : COLOR DETECTION AND ANALYSIS (WEEK 9)

LECTURER'S NAME: ASSOC. PROF. IR. TS. DR. ZULKIFLI BIN ZAINAL ABIDIN

DATE OF EXPERIMENT : 10 DECEMBER 2025

DATE OF SUBMISSION : 17 DECEMBER 2025

GROUP : 4

NO.	NAME	MATRIC NO.
1	MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	2311779
2	AHMAD HARITH IMRAN BIN MOHD YUSOF	2311581
3	ARIFA AQILAH BINTI ABDUL HALIM	2311530

ACKNOWLEDGEMENTS

We would like to express our heartfelt appreciation to Dr. Zulkifli Bin Zainal Abidin for his continuous guidance, support, and insightful explanations throughout this experiment. His expertise greatly helped us understand the concepts of interfacing and digital control using Arduino. We would also like to thank our lab assistant for providing technical assistance and ensuring that the experiment ran smoothly. Finally, we extend our gratitude to our classmates for their cooperation and teamwork during the lab session.

ABSTRACT

This experiment aimed to design and evaluate a color detection system using an ESP32 integrated with the Gravity HuskyLens AI Vision Camera. The study focused exclusively on an AI-based machine vision approach utilizing the HuskyLens built-in Color Recognition mode.

The HuskyLens was interfaced with the ESP32 through an I2C connection to enable efficient data communication. The camera was calibrated to recognize and store the characteristics of three distinct colored objects. Using the DFRobot_HuskyLens library, the ESP32 program requested detection data from the camera and retrieved the identified color ID and object coordinates. System performance was evaluated by repeatedly presenting the trained colors to the camera and monitoring detection results via the Serial Monitor.

The results showed that the system achieved reliable and consistent color recognition under stable lighting conditions, with fast response times after calibration. However, detection accuracy decreased when lighting conditions varied significantly. The experiment concludes that the HuskyLens, when integrated with the ESP32, provides an effective and user-friendly solution for color detection, highlighting the importance of proper calibration and controlled lighting. Potential improvements for future work, such as more robust calibration or advanced algorithms, are also discussed.

TABLE OF CONTENT

ABSTRACT.....	3
TABLE OF CONTENT.....	4
1.0 INTRODUCTION.....	5
2.0 MATERIAL AND EQUIPMENT.....	6
3.0 EXPERIMENTAL SETUP.....	7
4.0 METHODOLOGY.....	8
5.0 DATA COLLECTION.....	14
6.0 DATA ANALYSIS.....	15
7.0 RESULTS.....	16
8.0 DISCUSSION.....	17
9.0 CONCLUSION.....	19
10.0 RECOMMENDATIONS.....	20
11.0 REFERENCES.....	21
12.0 APPENDICES.....	22
13.0 STUDENT’S DECLARATION.....	25

1.0 INTRODUCTION

This experiment investigates the use of an Artificial Intelligence (AI) based machine vision system for color detection by integrating the Gravity HuskyLens AI Vision Camera with an ESP32 microcontroller. The primary purpose is to evaluate the capability of the HuskyLens to accurately and consistently detect and distinguish between three pre-calibrated colored objects. The system is configured using the HuskyLens built-in Color Recognition mode and communicates with the ESP32 via the I2C protocol.

Color detection is a fundamental concept in mechatronics and computer vision, with applications in automated sorting, quality inspection, and robotic navigation. Conventional methods typically rely on dedicated color sensors that measure reflected red, green, and blue (RGB) light intensities. In contrast, the HuskyLens employs onboard AI algorithms and image processing to recognize trained colors directly from visual input, providing detection data such as color ID and object coordinates. Communication between the HuskyLens and the ESP32 is achieved through I2C, allowing fast and reliable data transfer.

It is hypothesized that when properly calibrated, the HuskyLens will demonstrate high accuracy and consistency in detecting the learned colors under stable lighting conditions. However, variations in lighting or object color shades may reduce detection accuracy and system reliability.

2.0 MATERIAL AND EQUIPMENT

- ESP32
- Jumper wires
- Gravity HuskyLens AI Vision Camera
- RGB LED
- Computer with Arduino IDE installed
- USB cable for Arduino

3.0 EXPERIMENTAL SETUP

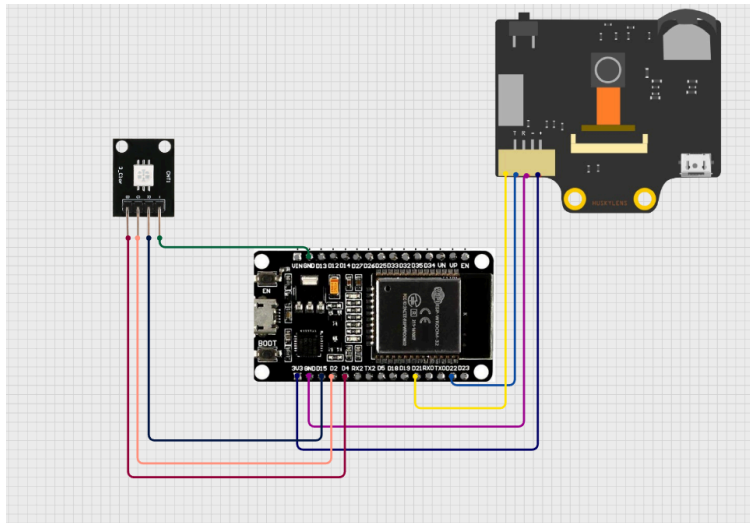


Figure A: Components connected to ESP32 via Cirkuit Designer

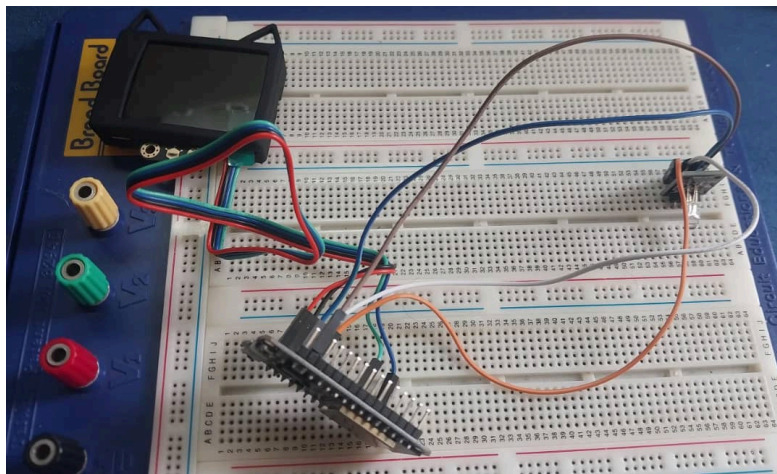


Figure B: Components connected to ESP32

1. The pin is connected as below;
 - RGB light module connection (B to 4, G to 2, R to 15 and '-' to GND)
 - Huskylens connection (VCC to 3.3v, GND to GND, SCL to 22 and SDA to 21)
2. Run the Arduino IDE program
3. Simulate the program using the coding

4.0 METHODOLOGY

1. The circuit was assembled according to the hardware requirements for the ESP32 and Gravity HuskyLens AI Vision Camera.
2. The HuskyLens was calibrated using its built-in Color Recognition mode to learn three different colored objects, assigning a unique ID to each.
3. The RGB LED was connected to the ESP32 to visually indicate the detected color ID.
4. The ESP32 was programmed using the Arduino IDE with the Wire library for I2C communication and the DFRobot_HuskyLens library to interface with the camera.
5. The code was uploaded to the ESP32, which then continuously requested detection data from the HuskyLens.
6. Detected color IDs were checked against the physical objects, and the corresponding RGB LED output was observed.
7. The process was repeated multiple times to verify accuracy, consistency, and response time under different lighting conditions.

Circuit Assembly

- The HuskyLens was connected to the ESP32 using I2C communication:
 - SDA → GPIO 21
 - SCL → GPIO 22
 - VCC → 5V
 - GND → GND
- The RGB LED was connected to ESP32 GPIO pins:
 - Red → GPIO 15
 - Green → GPIO 2
 - Blue → GPIO 4
- Power and ground connections were double-checked to ensure stable operation.

Programming Logic

- The ESP32 code initializes I2C communication and the HuskyLens using `huskylens.begin(Wire)`.
- The system continuously requests detection data from the HuskyLens (`huskylens.request()`) and checks if results are available (`huskylens.available()`).
- The detected color ID is read, and a `switch-case` structure maps each ID to the corresponding RGB LED color:
 - ID 1 → Red LED (`setColor(255,0,0)`)
 - ID 2 → Green LED (`setColor(0,255,0)`)
 - ID 3 → Blue LED (`setColor(0,0,255)`)
- The Serial Monitor outputs detection data, including object coordinates and ID, for verification.
- The system loops continuously, updating the RGB LED in real-time to reflect the detected color.

Code Used

```
#include "HUSKYLENS.h"      // Include HuskyLens library
#include <Wire.h>             // Include I2C library

HUSKYLENS huskylens;        // Create HuskyLens object

// --- RGB LED PIN DEFINITIONS (ESP32) ---
const int RED_PIN   = 15;    // Red LED pin
const int GREEN_PIN = 2;     // Green LED pin
const int BLUE_PIN  = 4;     // Blue LED pin

// Function to set RGB LED color
void setColor(int red, int green, int blue) {
  analogWrite(RED_PIN, red);  // Set red channel
  analogWrite(GREEN_PIN, green); // Set green channel
  analogWrite(BLUE_PIN, blue); // Set blue channel
}

// Function to print HuskyLens detection results
void printResult(HUSKYLENSResult result);
```

```

void setup() {
    Serial.begin(115200);    // Start Serial Monitor
    pinMode(RED_PIN, OUTPUT); // Set RED pin as output
    pinMode(GREEN_PIN, OUTPUT); // Set GREEN pin as output
    pinMode(BLUE_PIN, OUTPUT); // Set BLUE pin as output
    setColor(0, 0, 0);      // Turn off LED initially

    Wire.begin(21, 22);     // Initialize I2C bus (SDA=21, SCL=22)

    // Retry HuskyLens init if failed
    while (!huskylens.begin(Wire)) {
        Serial.println(F("Begin failed!"));
        Serial.println(F("1. Check \"Protocol Type\" in HuskyLens (I2C)"));
        Serial.println(F("2. Check the connection."));
        delay(700);
    }
}

```

```

void loop() {
    if (!huskylens.request()) { // Request frame analysis
        Serial.println(F("Fail to request data!"));
    }
    else if (!huskylens.isLearned()) { // Check if colors learned
        Serial.println(F("Nothing learned! Press learn button."));
        setColor(0, 0, 0); // Turn off LED
    }
    else if (!huskylens.available()) { // Check if object detected
        Serial.println(F("No block or arrow detected!"));
        setColor(0, 0, 0); // Turn off LED
    }
    else {
        Serial.println(F("#####")); // Separator
        while (huskylens.available()) {
            HUSKYLENSResult result = huskylens.read(); // Read detection
            printResult(result); // Print detection

            if (result.command == COMMAND_RETURN_BLOCK ||
                result.command == COMMAND_RETURN_ARROW) {

```

```

        switch (result.ID) { // Map ID to LED color
            case 1:
                Serial.println(F(">> ID 1: Red"));
                setColor(255, 0, 0); // RED
                break;
            case 2:
                Serial.println(F(">> ID 2: Green"));
                setColor(0, 255, 0); // GREEN
                break;
            case 3:
                Serial.println(F(">> ID 3: Blue"));
                setColor(0, 0, 255); // BLUE
                break;
        }
    }
}
}
}

```

```

// Print detected block or arrow info
void printResult(HUSKYLENSResult result){
    if (result.command == COMMAND_RETURN_BLOCK){
        Serial.println(String()+F("Block:xCenter=")+result.xCenter+
            F(",yCenter=")+result.yCenter+F(",width=")+result.width+
            F(",height=")+result.height+F(",ID=")+result.ID);
    }
    else if (result.command == COMMAND_RETURN_ARROW){
        Serial.println(String()+F("Arrow:xOrigin=")+result.xOrigin+
            F(",yOrigin=")+result.yOrigin+F(",xTarget=")+result.xTarget+
            F(",yTarget=")+result.yTarget+F(",ID=")+result.ID);
    }
    else{
        Serial.println("Object unknown!"); // Unknown detection
    }
}
}

```

Control Algorithm

1. Pin Setup

- The RGB LED channels are connected to the ESP32:
 - Red → GPIO 15
 - Green → GPIO 2
 - Blue → GPIO 4
- The HuskyLens is connected to the ESP32 via I2C:
 - SDA → GPIO 21
 - SCL → GPIO 22

2. Variable Declaration

- `HUSKYLENS huskylens` : stores the HuskyLens object for communication.
- `HUSKYLENSResult result` : stores the latest detection data from the camera.
- `RED_PIN, GREEN_PIN, BLUE_PIN` : store the GPIO numbers for the RGB LED.

3. Setup Function

- Initialize Serial communication with `Serial.begin(115200)` to monitor detection results.
- Configure RGB LED pins as outputs using `pinMode()`.
- Initialize the I2C bus with `Wire.begin(21, 22)`.
- Initialize HuskyLens with `huskylens.begin(Wire)` and retry until successful.
- Turn off the RGB LED initially with `setColor(0, 0, 0)`.

4. Main Loop

- Continuously request data from the HuskyLens using `huskylens.request()`.
- Check if the camera has learned colors with `huskylens.isLearned()`.
- Check for detected objects with `huskylens.available()`.
- If a detection exists, read it with `huskylens.read()` and store in `result`.

- Map the detected `result.ID` to the corresponding LED color using a `switch-case` structure:
 - ID 1 → Red LED (`setColor(255, 0, 0)`)
 - ID 2 → Green LED (`setColor(0, 255, 0)`)
 - ID 3 → Blue LED (`setColor(0, 0, 255)`)
 - Output detection data (coordinates, ID, width, height) to the Serial Monitor.
 - If no detection is available, turn off the RGB LED (`setColor(0, 0, 0)`).
5. LED Function (`setColor`)
- Sets the PWM value of each RGB LED channel according to the detected color.
 - Continuously updates the LED in real-time to reflect the currently detected color ID.
6. Result Printing Function (`printResult`)
- Checks whether the detected object is a block or an arrow.
 - Prints the coordinates, dimensions, and ID to the Serial Monitor.
 - If an unknown object is detected, prints "`Object unknown!`".

5.0 DATA COLLECTION

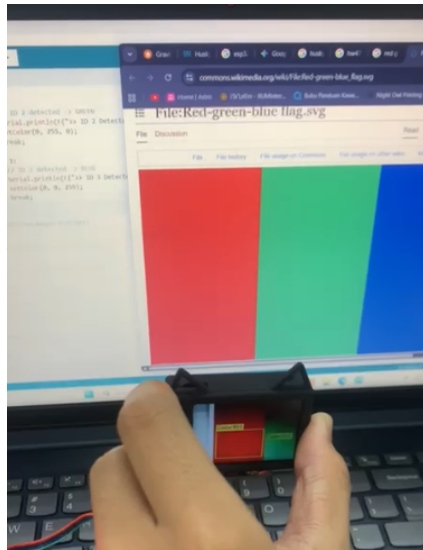


Figure C: Data collection on the HuskyLens

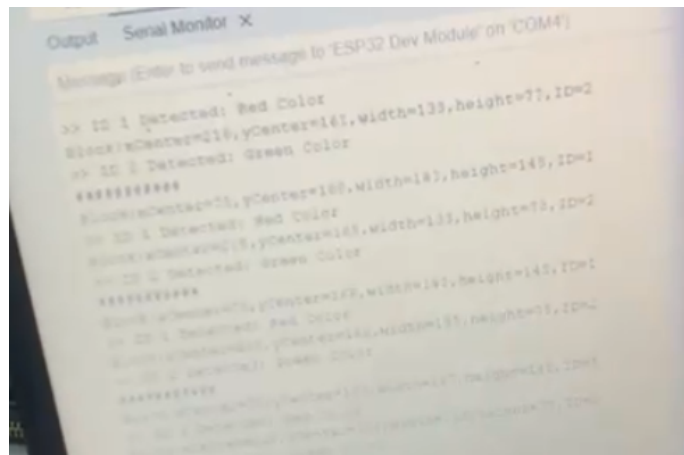


Figure D: Data collection on the serial monitor

The experiment involved using ESP32 with Huskylens and RGB light module connection. Below is the function of each components used in the experiment ;

#	Components	Function
1	ESP32 I2C communication	Able to retrieve data from the Huskylens
2	RGB light module	Project the colour of the light detect from Huskylens
3	Huskylens	Colour detection

6.0 DATA ANALYSIS

```

13:14:39.195 -> Block:xCenter=257,yCenter=92,width=36,height=48,ID=3
13:14:39.195 -> >> ID 3 Detected: Blue Color
13:14:39.195 -> #####
13:14:39.195 -> Block:xCenter=134,yCenter=170,width=49,height=50,ID=1
13:14:39.195 -> >> ID 1 Detected: Red Color
13:14:39.195 -> Block:xCenter=135,yCenter=90,width=51,height=50,ID=1
13:14:39.195 -> >> ID 1 Detected: Red Color
13:14:39.228 -> Block:xCenter=194,yCenter=162,width=45,height=33,ID=2
13:14:39.228 -> >> ID 2 Detected: Green Color
13:14:39.228 -> Block:xCenter=254,yCenter=92,width=41,height=47,ID=3
13:14:39.228 -> >> ID 3 Detected: Blue Color
13:14:39.228 -> #####
13:14:39.228 -> Block:xCenter=134,yCenter=170,width=49,height=49,ID=1
13:14:39.228 -> >> ID 1 Detected: Red Color
13:14:39.228 -> Block:xCenter=135,yCenter=90,width=51,height=51,ID=1
13:14:39.228 -> >> ID 1 Detected: Red Color
13:14:39.260 -> Block:xCenter=194,yCenter=163,width=45,height=33,ID=2
13:14:39.260 -> >> ID 2 Detected: Green Color
13:14:39.260 -> Block:xCenter=256,yCenter=93,width=37,height=48,ID=3
13:14:39.260 -> >> ID 3 Detected: Blue Color
13:14:39.260 -> #####
13:14:39.260 -> Block:xCenter=134,yCenter=170,width=49,height=49,ID=1
13:14:39.260 -> >> ID 1 Detected: Red Color
13:14:39.260 -> Block:xCenter=135,yCenter=90,width=51,height=51,ID=1
13:14:39.260 -> >> ID 1 Detected: Red Color

```

Figure E: Data collection in the serial monitor

From the Serial monitor, we can understand that the HuskyLens can detect three colour (Red, Green and Blue) and project the same colour onto the RGB light module. When one of the colour is detected, the RGB command on the Arduino IDE will input high at the desired light's colour.

7.0 RESULTS

The implementation and evaluation of the color detection system, integrated with the HuskyLens module, were successfully completed. An ESP32 microcontroller functioned as the central processing unit, utilizing an RGB LED as the visual output indicator. Data exchange between the ESP32 and the HuskyLens AI Vision Camera was established via the I2C communication protocol, enabling a stable connection that was verified upon system initialization. During experimental testing, the HuskyLens demonstrated the ability to accurately recognize the learned object colors and assign unique identification (ID) numbers to each.

The ESP32 controller retrieved these ID values and processed them using the specific control logic defined in the source code. When a valid color ID was detected, the system successfully drove the appropriate channel of the RGB LED to the active state. In particular, the detection of ID 1 resulted in the illumination of the Red LED, while ID 2 activated the Green LED, and ID 3 corresponded to the Blue LED output. Furthermore, the programmed reset logic effectively ensured that no color overlap occurred, maintaining only a single active LED color at any given time. System verification was conducted in real-time through the Serial Monitor, which displayed both the detected Color ID and the spatial coordinates of the object. In summary, the results demonstrate that the system satisfied the functional objectives, effectively translating visual data from the HuskyLens into immediate and accurate RGB LED responses.

8.0 DISCUSSION

This experiment's primary objective was to evaluate whether the Gravity HuskyLens AI Vision Camera could provide accurate and consistent color detection when interfaced with an ESP32 and an RGB LED as a physical output. As can be seen from the results, the system functioned as intended, in which the HuskyLens reliably detected the three trained colors under controlled lighting conditions, and the corresponding RGB LED illuminated in the correct color. This demonstrates the successful integration of the AI vision processing, I2C communication, and actuator control, with near-instantaneous response time suitable for real-time applications such as sorting or tracking.

Sources of Error and Limitations

1. Calibration Dependency

- The color IDs learned during the training phase are sensitive to ambient lighting. Changes in illumination or spectral content can reduce detection accuracy, as the HuskyLens stores a model based on the lighting conditions present during calibration.

2. Ambient Lighting Variation

- Variations in shadows or light color temperature directly affected detection performance. This was the primary external limitation of the system, highlighting the need for consistent lighting during operation.

3. Calibration Error

- Errors in the initial color learning process, such as quickly presenting the object or partial shadows during training, resulted in less robust color models and occasional misdetections.

Improvements

1. Reduce Effects of Ambient Lighting

- Add a dedicated white LED and shield the detection area to ensure consistent lighting.
 - Convert RGB data to HSV color space, as hue is less sensitive to brightness changes.
2. Improve Calibration Accuracy
 - Train the system under different lighting conditions and viewing angles.
 - Ensure the object is stationary, fully visible, and evenly lit during calibration.
 3. Apply Advanced Processing Techniques
 - Use adaptive white balancing to compensate for changes in ambient light color.
 - Continuously correct lighting effects before performing color detection.

9.0 CONCLUSION

In conclusion, the design and implementation of a color detection system utilizing the HuskyLens AI camera and an ESP32 microcontroller were successfully completed. The system achieved accurate color ID identification and data interpretation through the robust I2C communication link established between the vision sensor and the microcontroller.

Experimental results confirmed that every recognized color ID was correctly correlated with its specific RGB LED channel, providing immediate and distinct visual indicators. The implemented control logic effectively eliminated potential color overlap and ensured consistent system operation throughout the testing phase. Although the system displayed minor sensitivity to variations in ambient lighting, it successfully met all primary functional objectives with high responsiveness.

Overall, this project validates the effective integration of AI-based machine vision with ESP32-based control, establishing a solid framework for developing more complex mechatronic applications in the future.

10.0 RECOMMENDATIONS

To enhance the reliability of the color detection system in future iterations, it is strongly recommended to address the sensitivity to ambient lighting variations observed during the experiment. This can be achieved by integrating a dedicated, consistent light source, such as an LED ring attached directly to the HuskyLens, or by enclosing the detection area to minimize external light interference and reflections. Additionally, the calibration process should be improved by performing the color "learning" function at multiple angles and under the exact lighting conditions of the final deployment. This would ensure a more robust color model, significantly reducing the likelihood of false negatives when objects are slightly rotated or shadowed.

Furthermore, the system's control logic could be expanded to utilize the geometric data provided by the HuskyLens, specifically by implementing filters for object width and height to distinguish valid targets from background noise. For more advanced applications, future iterations should consider replacing the simple RGB LED output with active actuators, such as servo motors, to simulate real-world automation tasks like sorting objects into different bins based on the detected Color ID. These modifications would not only improve the system's robustness against environmental variables but also demonstrate a more practical and complex application of mechatronic system integration

11.0 REFERENCES

DfRobot. (n.d.). *HuskyLens series: Simple, smart, and scalable AI vision sensors*.

<https://www.dfrobot.com/huskylens>

Ahmed, O. (2022, January 20). Interfacing RGB LED with Arduino. Arduino Project Hub.

<https://projecthub.arduino.cc/semsemharaz/interfacing-rgb-led-with-arduino-b59902>

12.0 APPENDICES

Appendix A: Coding

```
1  #include "HUSKYLENS.h"
2  #include <Wire.h>
3
4  HUSKYLENS huskylens;
5
6  // --- RGB LED PIN DEFINITIONS (ESP32) ---
7  const int RED_PIN   = 15;
8  const int GREEN_PIN = 2;
9  const int BLUE_PIN  = 4;
10
11  void setColor(int red, int green, int blue) {
12      analogWrite(RED_PIN, red);
13      analogWrite(GREEN_PIN, green);
14      analogWrite(BLUE_PIN, blue);
15  }
16  |
17  void printResult(HUSKYLENSResult result);
18
19  void setup() {
20      Serial.begin(115200);
21
22      // Initialize RGB Pins
23      pinMode(RED_PIN, OUTPUT);
24      pinMode(GREEN_PIN, OUTPUT);
25      pinMode(BLUE_PIN, OUTPUT);
26
27      // Turn off LED initially
28      setColor(0, 0, 0);
29
30      Wire.begin(21, 22);
```

```

31     while (!huskylens.begin(Wire)) {
32         Serial.println(F("Begin failed!"));
33         Serial.println(F("1.Please recheck the \"Protocol Type\" in HUSKYLENS (General Settings>>Protocol Type>>I2C)"));
34         Serial.println(F("2.Please recheck the connection."));
35         delay(700);
36     }
37 }
38
39 void loop() {
40     if (!huskylens.request()) {
41         Serial.println(F("Fail to request data from HUSKYLENS, recheck the connection!"));
42     }
43     else if (!huskylens.islearned()) {
44         Serial.println(F("Nothing learned, press learn button on HUSKYLENS to learn one!"));
45         setColor(0, 0, 0); // Turn off if nothing is learned
46     }
47     else if (!huskylens.available()) {
48         Serial.println(F("No block or arrow appears on the screen!"));
49         setColor(0, 0, 0); // Turn off if nothing is detected
50     }
51     else {
52         Serial.println(F("#####"));
53         while (huskylens.available()) {
54             HUSKYLENSResult result = huskylens.read();
55             printResult(result);
56
57             // --- COLOR LOGIC BASED ON ID ---
58             if (result.command == COMMAND_RETURN_BLOCK || result.command == COMMAND_RETURN_ARROW) {

```

```

59                 switch (result.ID) {
60                     case 1:
61                         // ID 1 detected -> RED
62                         Serial.println(F(">>> ID 1 Detected: Red Color"));
63                         setColor(255, 0, 0);
64                         break;
65
66                     case 2:
67                         // ID 2 detected -> GREEN
68                         Serial.println(F(">>> ID 2 Detected: Green Color"));
69                         setColor(0, 255, 0);
70                         break;
71
72                     case 3:
73                         // ID 3 detected -> BLUE
74                         Serial.println(F(">>> ID 3 Detected: Blue Color"));
75                         setColor(0, 0, 255);
76                         break;
77                 }
78             }
79         }
80     }
81 }
82
83 void printResult(HUSKYLENSResult result){
84     if (result.command == COMMAND_RETURN_BLOCK){
85         Serial.println(String()+F("Block:xCenter=")+result.xCenter+F(",yCenter=")+result.yCenter+F(",width=")+result.width+F(",height=")+result.height+F(",ID=")+result.ID);
86     }
87     else if (result.command == COMMAND_RETURN_ARROW){
88         Serial.println(String()+F("Arrow:xOrigin=")+result.xOrigin+F(",yOrigin=")+result.yOrigin+F(",xTarget=")+result.xTarget+F(",yTarget=")+result.yTarget+F(",ID=")+result.ID);
89     }
90     else{

```

```

91         Serial.println("Object unknown!");
92     }
93 }

```

Appendix B: Data collection

```
13:14:39.195 -> Block:xCenter=257,yCenter=92,width=36,height=48,ID=3
13:14:39.195 -> >> ID 3 Detected: Blue Color
13:14:39.195 -> #####
13:14:39.195 -> Block:xCenter=134,yCenter=170,width=49,height=50,ID=1
13:14:39.195 -> >> ID 1 Detected: Red Color
13:14:39.195 -> Block:xCenter=135,yCenter=90,width=51,height=50,ID=1
13:14:39.195 -> >> ID 1 Detected: Red Color
13:14:39.228 -> Block:xCenter=194,yCenter=162,width=45,height=33,ID=2
13:14:39.228 -> >> ID 2 Detected: Green Color
13:14:39.228 -> Block:xCenter=254,yCenter=92,width=41,height=47,ID=3
13:14:39.228 -> >> ID 3 Detected: Blue Color
13:14:39.228 -> #####
13:14:39.228 -> Block:xCenter=134,yCenter=170,width=49,height=49,ID=1
13:14:39.228 -> >> ID 1 Detected: Red Color
13:14:39.228 -> Block:xCenter=135,yCenter=90,width=51,height=51,ID=1
13:14:39.228 -> >> ID 1 Detected: Red Color
13:14:39.260 -> Block:xCenter=194,yCenter=163,width=45,height=33,ID=2
13:14:39.260 -> >> ID 2 Detected: Green Color
13:14:39.260 -> Block:xCenter=256,yCenter=93,width=37,height=48,ID=3
13:14:39.260 -> >> ID 3 Detected: Blue Color
13:14:39.260 -> #####
13:14:39.260 -> Block:xCenter=134,yCenter=170,width=49,height=49,ID=1
13:14:39.260 -> >> ID 1 Detected: Red Color
13:14:39.260 -> Block:xCenter=135,yCenter=90,width=51,height=51,ID=1
13:14:39.260 -> >> ID 1 Detected: Red Color
```

13.0 STUDENT'S DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

Signature: 	Read	/
Name: AHMAD HARITH IMRAN BIN MOHD YUSOF	Understand	/

Matric No.: 2311581	Agree	/
Contribution : Assembler, table of content, results, conclusion, recommendation and student declaration.		

Signature: 	Read	/
Name: MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	Understand	/
Matric No.: 2311779	Agree	/
Contribution : Coder, experimental setup, data collection, data analysis, references and appendices.		

Signature: 	Read	/
--	------	---

Name: ARIFA AQILAH BINTI ABDUL HALIM	Understand	/
Matric No.: 2311530	Agree	/
Contribution : Assembler, abstract, introduction,material and equipment, methodology and discussion.		